PromptZal IDE

Manual Técnico

1. Introducción

Este documento describe el diseño y la implementación del compilador del lenguaje **PromptZal**. El compilador recibe un archivo `.pz` y produce el **promptzal**: un prompt en texto (Markdown) listo para usarse en Claude Code o en cualquier herramienta de IA.

El proyecto completa las cuatro fases del compilador:

Fase	Implementación	¿Generador?
Análisis léxico	Especificación JFlex (`Lexer.flex`)	Sí, JFlex (obligatorio)
Análisis sintáctico	Parser descendente recursivo escrito a mano	No
AST, tabla de símbolos y semántica	Código Java propio	No
Generación del promptzal	Código Java propio	No

2. Entorno de desarrollo

Herramientas y dependencias

Componente	Versión	Uso
Java (JDK)	21	Lenguaje de implementación
Apache Maven	incluido en NetBeans	Construcción y dependencias
Apache NetBeans	21	IDE de desarrollo
jflex-maven-plugin	1.9.1	Genera `Lexer.java` desde `Lexer.flex` en cada build
JavaFX (javafx-controls)	21.0.5	Interfaz gráfica
javafx-maven-plugin	0.0.8	Ejecución con `mvn clean javafx:run`
RichTextFX	0.11.2	Editor de código con estilos por rango y números de línea
AtlantaFX	2.0.1	Tema visual (PrimerDark)
Git / GitHub	—	Control de versiones

Integración de JFlex con Maven

El plugin `jflex-maven-plugin` busca las especificaciones en `src/main/jflex/` y, en la fase `generate-sources` de Maven, genera el analizador en `target/generated-sources/jflex/`. NetBeans reconoce esa carpeta como código fuente generado, por lo que la clase `Lexer` se usa como cualquier otra.

- El archivo versionado en Git es **solo** `Lexer.flex`; `Lexer.java` se regenera en cada **Clean and Build** y está excluido por `.gitignore` (carpeta `target/`).
- El paquete del lexer generado lo define la primera línea del `.flex`: `package com.mycompany.promtzalp2.lexer;`.

```
<plugin>
    <groupId>de.jflex</groupId>
    <artifactId>jflex-maven-plugin</artifactId>
    <version>1.9.1</version>
    <executions>
        <execution>
            <goals><goal>generate</goal></goals>
        </execution>
    </executions>
</plugin>
```

Compilación y ejecución

Acción	Comando / configuración
Construir	**Clean and Build** (equivale a `mvn clean install`)
Ejecutar	**Run (F6)**, configurado en Properties → Actions → Run Project con el goal `clean javafx:run` (guardado en `nbactions.xml`)
Clase de arranque	`com.mycompany.promtzalp2.gui.App` (extiende `javafx.application.Application`)

Estructura de carpetas

```
PromtZalP2/
├── pom.xml                       dependencias y plugins
├── nbactions.xml                 configuracion de Run (javafx:run)
├── pruebas/                      casos de prueba .pz
├── reportes/                     reportes HTML generados
└── src/main/
    ├── jflex/Lexer.flex          especificacion del analizador lexico
    ├── resources/estilos.css     colores del editor e interfaz
    └── java/com/mycompany/promtzalp2/
        ├── PromtZalP2.java       main: inicia la interfaz
        ├── Compilador.java       orquesta las 4 fases
        ├── ResultadoCompilacion.java
        ├── lexer/                Token, TipoToken
        ├── errores/              TipoError, ErrorCompilacion, ManejadorErrores
        ├── parser/               Parser
        ├── ast/                  NodoAST y nodos del arbol
        ├── semantico/            TipoSimbolo, Simbolo, TablaSimbolos, AnalizadorSemantico
        ├── generador/            GeneradorPromptzal
        ├── reportes/             PlantillaHTML y reportes
        └── gui/                  App, VentanaPrincipal, editor y paneles
```

3. Arquitectura del compilador

Flujo de compilación

La clase `Compilador` ejecuta las fases en orden y entrega un objeto `ResultadoCompilacion` con todo lo producido: tokens, AST, tabla de símbolos, errores, promptzal y tiempo. Todas las fases registran sus errores en un único `ManejadorErrores`, lo que permite mostrarlos unificados en la interfaz y en el reporte.

```
texto .pz
   │
   ├─► Lexer (pasada 1) ─► tokens + errores lexicos
   │
   └─► Lexer (pasada 2) ─► Parser ─► AST + errores sintacticos
                                        │
                                        ├─► AnalizadorSemantico
                                        │       ─► TablaSimbolos + errores semanticos
                                        │
                                        └─► GeneradorPromptzal ─► promptzal
```

El lexer se ejecuta dos veces: la primera pasada construye la lista de tokens (para el panel y el reporte) y registra los errores léxicos; la segunda alimenta al parser con el manejador de errores en `null`, para no duplicar los errores léxicos.

Paquetes

Paquete	Responsabilidad
`com.mycompany.promtzalp2`	Punto de entrada, `Compilador` y `ResultadoCompilacion`.
`lexer`	Tokens y su clasificación. El `Lexer` lo genera JFlex.
`errores`	Representación y registro de errores de las tres fases.
`parser`	Análisis sintáctico descendente recursivo con modo pánico.
`ast`	Nodos del árbol de sintaxis abstracta.
`semantico`	Tabla de símbolos y validaciones semánticas.
`generador`	Redacción del promptzal a partir del AST.
`reportes`	Reportes HTML.
`gui`	Interfaz gráfica tipo IDE (JavaFX).

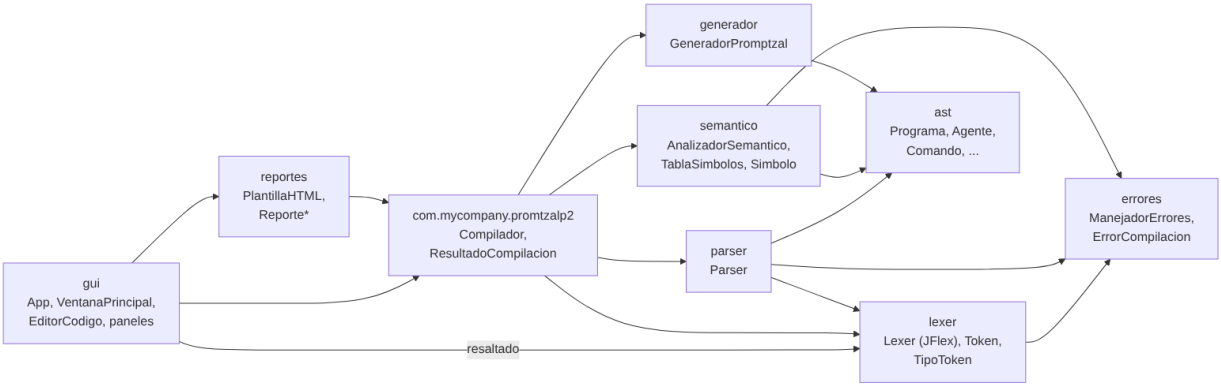


Figura 1. Diagrama de paquetes y sus dependencias

4. Análisis léxico (JFlex)

Tabla de terminales

Cada tipo de token es un valor del enum `TipoToken` y corresponde a un terminal de la gramática.

Terminal	Lexemas que lo producen	Categoría
DIRECTIVA	`@modelo`, `@rol`, `@formato`	Directiva
AGENTE	`AGENTE`	Palabra reservada
CONTEXTO	`contexto`	Palabra reservada
VARIABLE	`variable`	Palabra reservada
EJECUTAR	`EJECUTAR`	Palabra reservada
EXPORTAR	`EXPORTAR`	Palabra reservada
COMANDO_IA	`PREGUNTAR`, `GENERAR`, `RESUMIR`, `ANALIZAR`, `TRADUCIR`, `CLASIFICAR`, `EXTRAER`	Comando de IA
CARGAR	`CARGAR`	Función del sistema
CONECTOR	`SOBRE`, `DESDE`, `EN`, `COMO`	Conector
FLECHA	`->`	Asignación de resultado
IGUAL	`=`	Operador
MAS	`+`	Operador
LLAVE_A / LLAVE_C	`{` / `}`	Delimitador
PAR_A / PAR_C	`(` / `)`	Delimitador
COMA	`,`	Delimitador
ID	Letra o `_`, seguido de letras, dígitos o `_`	Identificador
CADENA	Texto entre comillas dobles en una sola línea	Literal
NUMERO	Enteros (`10`) y decimales (`0.7`)	Literal
EOF	Fin del archivo (generado por el lexer)	Control
COMENTARIO	`// ...` y `/* ... */` (solo modo resaltado)	Interno del editor
ERROR	Lexema inválido (solo modo resaltado)	Interno del editor

> **Nota:** `COMENTARIO` y `ERROR` nunca llegan al parser. Solo los emite el lexer en modo resaltado, para que el editor pueda colorearlos.

Estructura del archivo Lexer.flex

Sección	Contenido
Código de usuario (antes del primer %%)	Declaración de `package` e `import` de las clases de errores.
Opciones y declaraciones (entre %% y %%)	Opciones de JFlex, código auxiliar en `%{ %}`, valor de EOF en `%eofval{ %eofval}`, estados y macros.
Reglas léxicas (después del segundo %%)	Pares **patrón { acción }** agrupados por estado.

Opciones utilizadas

Opción	Efecto
`%class Lexer` / `%public`	Nombre y visibilidad de la clase generada.
`%unicode`	Soporte de todo el rango Unicode.
`%line` / `%column`	Habilita `yyline` y `yycolumn` (desde 0; se suma 1 al crear el token).
`%char`	Habilita `yychar`: posición absoluta en el texto, usada por el resaltado del editor.
`%type Token`	`yylex()` devuelve objetos `Token`.
`%state COMENTARIO_BLOQUE`	Estado léxico para reconocer comentarios de bloque.

Macros (expresiones regulares nombradas)

```
LETRA            = [a-zA-Z_]
DIGITO           = [0-9]
ID               = {LETRA}({LETRA}|{DIGITO})*
ENTERO           = {DIGITO}+
DECIMAL          = {DIGITO}+ "." {DIGITO}+
CADENA           = \" [^\"\r\n]* \"
CADENA_ABIERTA   = \" [^\"\r\n]*
COMENTARIO_LINEA = "//" [^\r\n]*
SALTO            = \r | \n | \r\n
ESPACIO          = [ \t\f]
```

Código auxiliar

Miembro	Función
`Lexer(Reader, ManejadorErrores, boolean)`	Constructor adicional: recibe dónde registrar errores y si trabaja en modo resaltado.
`token(TipoToken)`	Crea el `Token` con lexema, fila y columna (desde 1), offset y longitud.
`registrarError(...)`	Agrega un `ErrorCompilacion` de tipo LEXICO al manejador (si no es `null`).
`error(String)`	Registra el error; devuelve un token ERROR en modo resaltado o `null` en modo compilación.

Reglas de reconocimiento

- **Longest match:** JFlex elige siempre el lexema más largo. Por eso `ENTREGAR` es un único ID y no `EN` + `TREGAR`, y `->` es FLECHA y no un error seguido de `>`.
- **Prioridad por orden:** si dos reglas reconocen la misma longitud, gana la primera. Las palabras reservadas, comandos y conectores se declaran **antes** de `{ID}`.
- **Cadenas:** si la cadena cierra, `{CADENA}` es más larga que `{CADENA_ABIERTA}` y gana. Si llega al salto de línea sin cerrar, solo aplica `{CADENA_ABIERTA}`, que se reporta como error.
- **Directivas desconocidas:** la regla `"@" {ID}` (después de las directivas válidas) captura casos como `@color` y los reporta.
- **Comentarios de bloque:** al leer `/*` se guarda la posición inicial y se pasa al estado `COMENTARIO_BLOQUE`, donde se acumula texto hasta `*/`. Si se alcanza el fin del archivo, se reporta "Comentario de bloque sin cerrar".
- **Cualquier otro carácter:** la regla final `[^]` captura todo carácter no reconocido, lo reporta y el análisis continúa con el siguiente.

Errores léxicos

Situación	Mensaje	Recuperación
Carácter fuera del alfabeto	Caracter no reconocido	Se descarta ese único carácter.
Comillas sin cerrar en la línea	Cadena sin cerrar	Se descarta hasta el fin de la línea.
`@` seguido de nombre no válido	Directiva no reconocida	Se descarta el lexema completo.
`/*` sin `*/` antes de EOF	Comentario de bloque sin cerrar	Se termina el análisis (EOF).

En todos los casos el error se registra con lexema, descripción, fila y columna, y el análisis **no se detiene**.

Modo compilación y modo resaltado

Aspecto	Compilación (`false`)	Resaltado (`true`)
Comentarios	Se descartan	Se devuelven como COMENTARIO
Errores	Se registran y se descartan	Se devuelven como ERROR
Usado por	`Compilador` y `Parser`	`ResaltadorSintaxis` (editor)

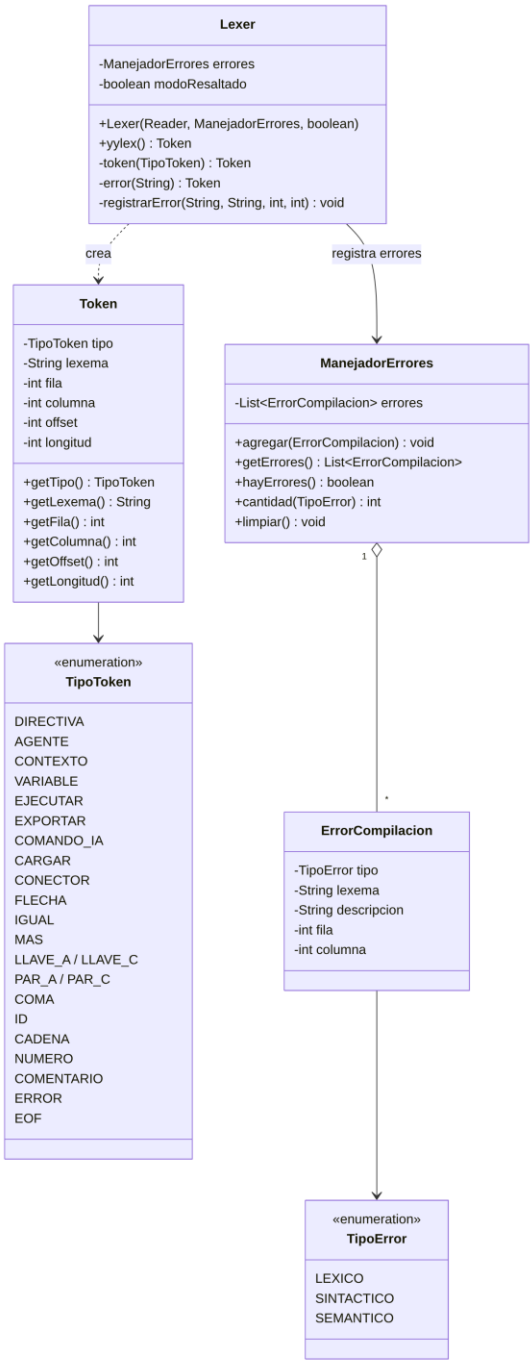


Figura 2. Diagrama de clases: lexer y errores.

5. Análisis sintáctico

Gramática libre de contexto (BNF)

Se implementa la gramática de referencia del enunciado. No tiene recursividad por la izquierda y está factorizada, por lo que es LL(1): con un solo token de anticipación se decide qué regla aplicar.

```
<programa>        ::= <directivas> <agentes> <ejecuciones> <exportar> EOF
<directivas>      ::= DIRECTIVA CADENA <directivas> | ε
<agentes>         ::= <agente> <mas_agentes>
<mas_agentes>     ::= <agente> <mas_agentes> | ε
<agente>          ::= AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C
<contexto>        ::= CONTEXTO IGUAL CADENA
<variables>       ::= VARIABLE ID IGUAL <expresion> <variables> | ε
<comandos>        ::= <comando> <mas_comandos>
<mas_comandos>    ::= <comando> <mas_comandos> | ε
<comando>         ::= COMANDO_IA <expresion> <conectores> FLECHA ID
<conectores>      ::= CONECTOR <expresion> <conectores> | ε
<expresion>       ::= <termino> <mas_terminos>
<mas_terminos>    ::= MAS <termino> <mas_terminos> | ε
<termino>         ::= CADENA | ID | NUMERO <unidad> | <cargar>
<unidad>          ::= ID | ε
<cargar>          ::= CARGAR PAR_A CADENA PAR_C
<ejecuciones>     ::= EJECUTAR ID <mas_ejecuciones>
<mas_ejecuciones> ::= EJECUTAR ID <mas_ejecuciones> | ε
<exportar>        ::= EXPORTAR ID <mas_ids>
<mas_ids>         ::= COMA ID <mas_ids> | ε
```

Correspondencia no terminal → método

Cada no terminal es un método privado de la clase `Parser`; cada método lleva su regla como comentario Javadoc.

No terminal	Método	Construye
<programa>	`programa()`	`Programa` (raíz)
<directivas>	`directivas()`	`Directiva`
<agentes> / <mas_agentes>	`agentes()` / `masAgentes()`	—
<agente>	`agente()`	`Agente`
<contexto>	`contexto(Agente)`	`Termino` (CADENA)
<variables>	`variables(Agente)`	`Variable`
<comandos> / <mas_comandos>	`comandos(Agente)` / `masComandos(Agente)`	—
<comando>	`comando(Agente)`	`Comando`
<conectores>	`conectores(Comando)`	`Conector`
<expresion> / <mas_terminos>	`expresion()` / `masTerminos(Expresion)`	`Expresion`
<termino>	`termino()`	`Termino`

No terminal	Método	Construye
<unidad>	`unidad()`	unidad del número
<cargar>	`cargar()`	`Termino` (CARGAR)
<ejecuciones> / <mas_ejecuciones>	`ejecuciones()` / `masEjecuciones()`	`Referencia`
<exportar> / <mas_ids>	`exportar()` / `masIds()`	`Referencia`

Métodos auxiliares

Método	Función
`avanzar()`	Pide el siguiente token al lexer (`yylex()`).
`verificar(tipo)`	Indica si el token actual es del tipo dado, sin consumirlo. Decide las alternativas ε.
`consumir(tipo, esperado)`	Si coincide, consume y devuelve el token; si no, reporta el error y lanza `ErrorSintactico`.
`reportarError(esperado)`	Registra el error con lo esperado y lo encontrado, fila y columna.
`sincronizar()`	Descarta tokens hasta uno de sincronización (modo pánico).
`sinComillas(lexema)`	Quita las comillas de una cadena antes de guardarla en el AST.

Recuperación de errores: modo pánico

Al encontrar un token inesperado, el parser (1) reporta el error indicando **token encontrado y esperado**, (2) descarta tokens hasta un token de sincronización y (3) continúa el análisis. La excepción interna `ErrorSintactico` sube desde el punto del error hasta el `try/catch` más cercano, que es el punto de recuperación de esa parte del programa.

Tokens de sincronización

Token	Motivo
AGENTE	Inicia un nuevo agente.
LLAVE_C	Cierra el agente actual.
COMANDO_IA	Inicia un nuevo comando.
VARIABLE	Inicia una nueva declaración.
EJECUTAR / EXPORTAR	Inician la sección final del programa.
CONTEXTO (agregado)	Inicia el cuerpo de un agente; evita perder el contexto tras un error en el encabezado.
DIRECTIVA (agregado)	Inicia una nueva directiva.

Token	Motivo
EOF	Fin del archivo; nunca se descarta.

Puntos de recuperación

Método	Error típico	Continúa en
`directivas()`	Directiva sin cadena	Siguiente directiva o AGENTE
`agente()` (encabezado)	Falta nombre o `{`	El contexto del mismo agente
`contexto()`	Contexto mal escrito o ausente	Variables o comandos
`variables()`	Declaración incorrecta	Siguiente variable o comando
`comando()`	Falta `->` o destino	Siguiente comando
`masComandos()`	Token ajeno dentro del agente	Se descarta y se buscan más comandos
`agente()` (cierre)	Falta `}`	Siguiente AGENTE o EJECUTAR
`ejecuciones()`	EJECUTAR sin nombre	Siguiente EJECUTAR o EXPORTAR
`exportar()`	Lista mal escrita	Fin del archivo

> **Terminación garantizada:** cada punto de recuperación consume al menos un token antes de fallar, o el token donde se detiene lo procesa otra regla. En `masComandos()` se llama a `avanzar()` antes de sincronizar. Así el modo pánico nunca entra en un ciclo infinito.

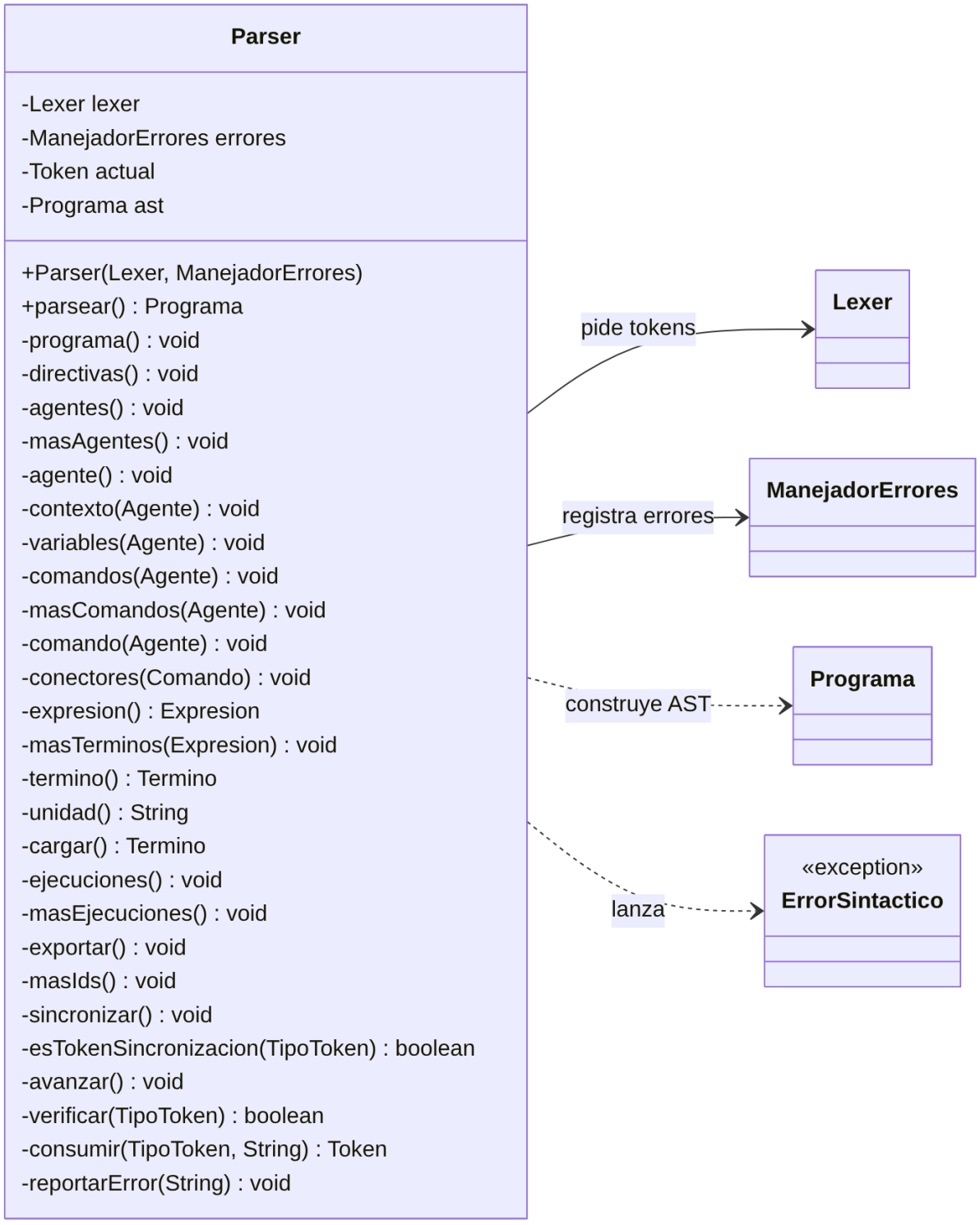


Figura 3. Diagrama de clases: Parser.

6. Árbol de sintaxis abstracta (AST)

Diseño

Todas las clases del AST heredan de la clase abstracta `NodoAST`, que guarda la fila y columna y obliga a implementar `getEtiqueta()` y `getHijos()`. Gracias a ello, el árbol se recorre con **una sola función recursiva**, que usan el `TreeView` de la interfaz y el reporte HTML.

Clase	Regla	Contenido
`Programa`	<programa>	Listas de directivas, agentes, ejecuciones y exportaciones
`Directiva`	DIRECTIVA CADENA	Nombre (`@modelo`) y valor sin comillas
`Agente`	<agente>	Nombre, contexto, variables y comandos
`Variable`	VARIABLE ID = <expresion>	Nombre y valor (`Expresion`)
`Comando`	<comando>	Nombre, argumento, conectores y destino
`Conector`	CONECTOR <expresion>	Nombre y argumento
`Expresion`	<expresion>	Lista de `Termino`
`Termino`	<termino>	Tipo (`TipoTermino`), valor y unidad opcional
`Referencia`	EJECUTAR ID / EXPORTAR ID	Palabra que la origina y nombre referido

Ejemplo

```
AGENTE analista {
  contexto = "Eres un analista"
  variable ventas = CARGAR("ventas.csv")
  RESUMIR ventas EN 100 palabras -> resumen
}
```

```
Programa
└── AGENTE analista
    ├── Cadena: "Eres un analista"
    ├── variable ventas
    │   └── Expresion
    │       └── CARGAR("ventas.csv")
    └── RESUMIR -> resumen
        ├── Expresion
        │   └── Id: ventas
        └── Conector: EN
            └── Expresion
                └── Numero: 100 palabras
```

El agente se agrega al programa en cuanto se lee su encabezado, y los comandos en cuanto se lee su nombre; así, aunque ocurra un error sintáctico más adelante, el AST conserva todo lo reconocido.

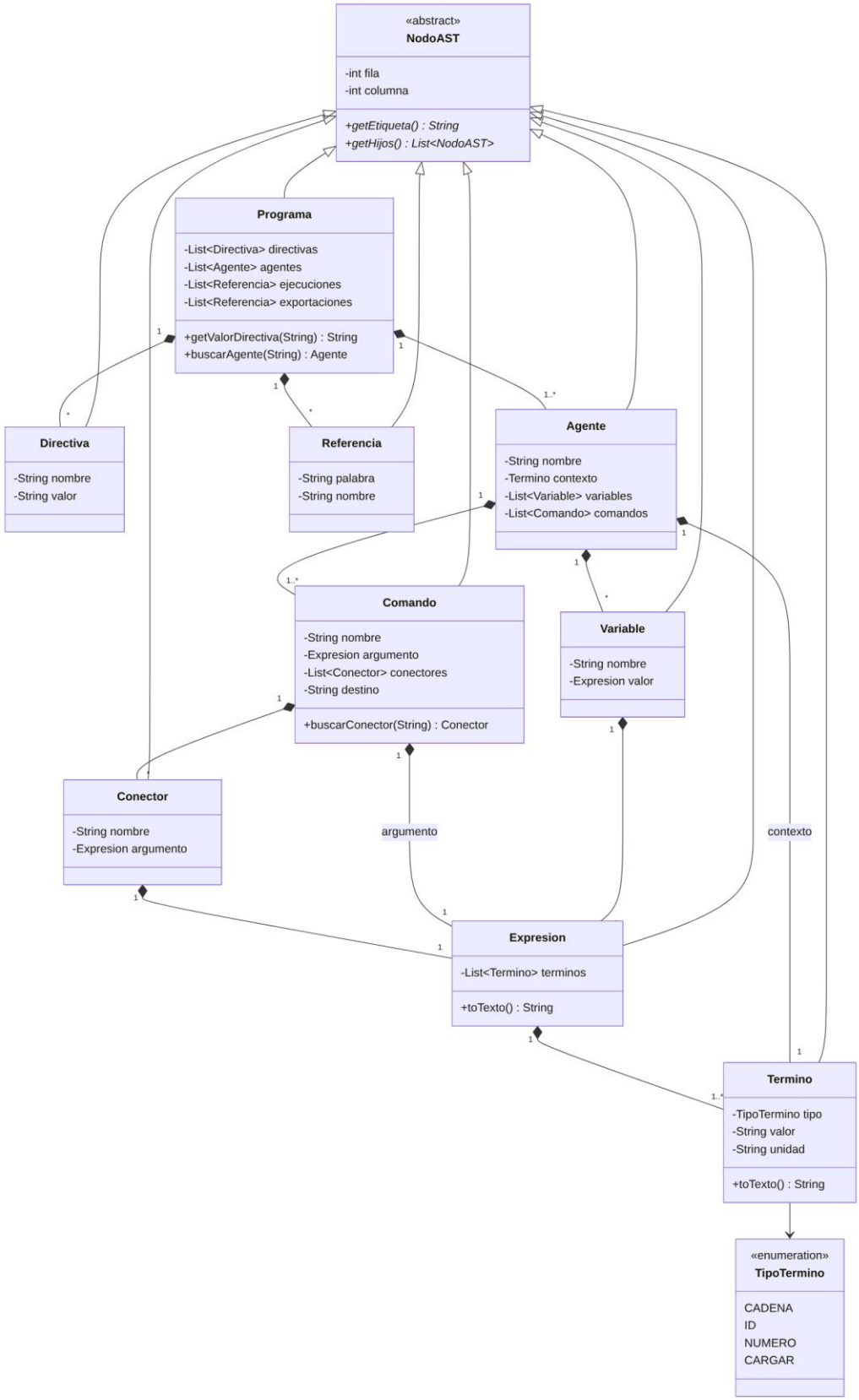


Figura 4. Diagrama de clases: AST.

7. Tabla de símbolos y análisis semántico

Tabla de símbolos

Campo de Simbolo	Descripción
nombre	Identificador del agente o variable.
tipo	`AGENTE`, `VARIABLE` (declarada con `variable`) o `RESULTADO` (destino de `->`).
ámbito	Nombre del agente al que pertenece; los agentes tienen ámbito `global`.
fila / columna	Posición de la declaración.

Los símbolos se agregan **mientras se recorre el programa en orden**, por lo que una búsqueda solo encuentra los declarados antes; así se cumple la regla de declarar antes de usar. `buscar(nombre, ambito)` solo devuelve variables de ese agente, lo que implementa los ámbitos separados.

Validaciones semánticas

#	Regla	Mensaje de error
1	Identificador usado en una expresión debe estar declarado antes, en el mismo agente	La variable 'x' no esta declarada antes de usarse en el agente 'a'
2	No puede haber dos agentes con el mismo nombre	El agente 'a' ya fue declarado en la fila n
3	EJECUTAR solo puede referirse a un agente existente	EJECUTAR se refiere al agente 'a', que no existe
4	EXPORTAR solo puede referirse a variables de algún agente ejecutado	EXPORTAR se refiere a 'x', que no existe en ningun agente ejecutado

Excepciones

- En `GENERAR`, el **primer identificador** del argumento indica el tipo de contenido (`GENERAR codigo`) y no se valida como variable (parámetro `primerIdEsTipo`).
- La **unidad** de un número (`palabras` en `100 palabras`) se guarda en el campo `unidad` del `Termino`, no como ID, por lo que no se valida.

Orden del análisis

1. Por cada agente: verificar duplicado y registrarlo (un agente duplicado no se analiza, para no mezclar ámbitos).
2. Por cada variable: validar su valor y **después** declararla (impide usarla en su propia definición).
3. Por cada comando: validar argumento y conectores, y **después** declarar el destino como RESULTADO.
4. Validar cada EJECUTAR y obtener el conjunto de agentes ejecutados válidos.
5. Validar cada EXPORTAR contra las variables de esos agentes.

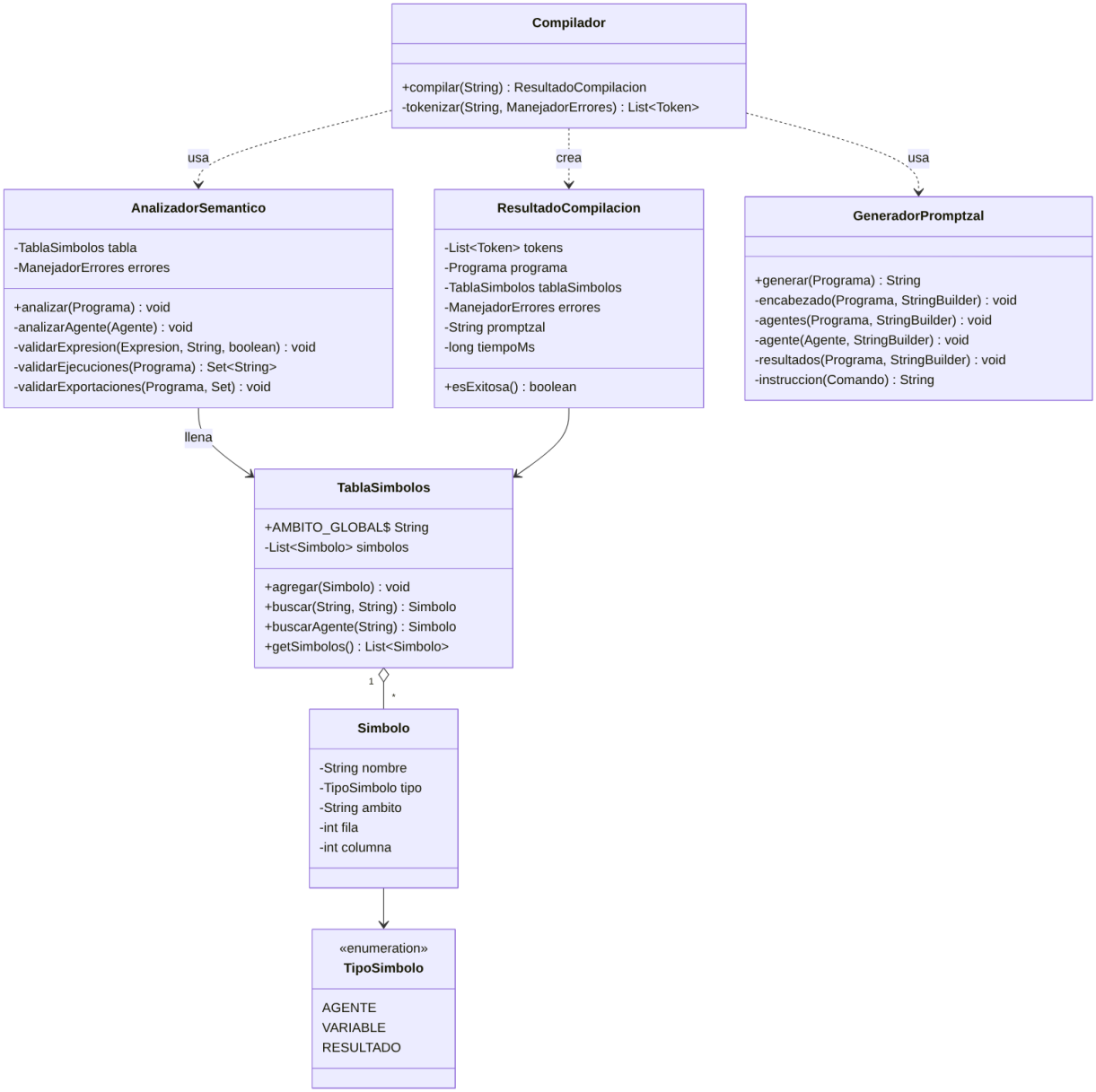


Figura 5. Diagrama de clases: compilador, semántico y generador.

8. Generación del promptzal

1. **Encabezado:** título y las directivas declaradas (modelo destino, rol general, formato de respuesta).
2. **Agentes:** por cada agente de EJECUTAR, en ese orden y sin repetir: contexto, variables con el origen de su valor (literal o archivo de CARGAR) e instrucciones numeradas.
3. **Resultado solicitado:** la lista de EXPORTAR.

Traducción de comandos

Comando PromptZal	Instrucción generada
PREGUNTAR p SOBRE x -> y	Basandote en x, responde: p. Guarda la respuesta como y.
GENERAR t DESDE e -> y	Genera t a partir de: e. Guarda el resultado como y.
RESUMIR x EN n -> y	Resume x en un maximo de n. Guarda el resultado como y.
ANALIZAR x SOBRE c -> y	Analiza x considerando: c. Guarda el resultado como y.
TRADUCIR x EN i -> y	Traduce x al idioma i. Guarda el resultado como y.
CLASIFICAR x EN c -> y	Clasifica x en las categorias: c. Guarda el resultado como y.
EXTRAER e DESDE x -> y	Extrae e a partir de x. Guarda el resultado como y.
... COMO f	Agrega: Entrega el resultado en formato f.
Comando nuevo sin caso	Realiza la accion NOMBRE sobre x. (instrucción genérica)

Ejemplo de salida

```
# Prompt generado con PromptZal

Modelo destino: claude-sonnet-4-6
Rol general: analista de datos
Formato de respuesta: markdown

## Agente: analista

Contexto: Eres un analista de datos experto

Variables disponibles:
- ventas: contenido del archivo "ventas.csv" (adjuntar el archivo)

Instrucciones:
1. Basandote en ventas, responde: Cuales son las 3 tendencias principales? Guarda la respuesta como
tendencias.
2. Genera codigo a partir de: grafica de barras de tendencias. Guarda el resultado como script.
3. Resume tendencias en un maximo de 100 palabras. Guarda el resultado como resumen.

## Resultado solicitado
- resumen
- script
```

El promptzal se muestra en el panel derecho de la interfaz y puede guardarse como `.md` o `.txt`. Si hubo errores, se genera con lo reconocido y se muestra un aviso.

9. Manejo de errores

Las tres fases comparten un único `ManejadorErrores`. Cada error es un `ErrorCompilacion` con tipo, lexema, descripción, fila y columna.

Fase	Formato de la descripción	Ejemplo
Léxica	Tipo de error léxico	Caracter no reconocido → `$`
Sintáctica	Se esperaba X pero se encontro Y	Se esperaba '->' pero se encontro 'tendencias'
Semántica	Descripción de la regla violada	EJECUTAR se refiere al agente 'revisor', que no existe

- Ninguna fase se detiene en el primer error: todos se reportan en una sola corrida.
- Un error léxico puede provocar uno sintáctico (por ejemplo, una cadena sin cerrar deja al parser sin el token esperado). Es el comportamiento esperado de un compilador por fases.
- En la interfaz, los errores se listan en la pestaña Problemas (ordenados por posición), se subrayan en el editor y se incluyen en el reporte HTML.

10. Interfaz gráfica

La interfaz se construyó con JavaFX, simulando un IDE moderno. `App` aplica el tema PrimerDark de AtlantaFX y carga `estilos.css`; `VentanaPrincipal` arma la distribución y conecta los eventos.

Clase	Responsabilidad
`App`	Arranque de JavaFX; entrega `HostServices` para abrir reportes en el navegador.
`VentanaPrincipal`	Menú, barra de herramientas, paneles, manejo de archivos `.pz`, compilación y reportes.
`EditorCodigo`	`CodeArea` de RichTextFX: números de línea, resaltado, marcas de error, navegación a fila/columna, control de cambios sin guardar.
`ResaltadorSintaxis`	Tokeniza el texto con el **mismo Lexer** en modo resaltado y asigna una clase CSS por token; superpone el subrayado de errores.
`ArbolAST`	`TreeView` del AST construido recursivamente; doble clic lleva a la línea del nodo.
`PanelPromptzal`	Muestra el prompt; copiar al portapapeles y guardar como `.md` o `.txt`.
`PanelProblemas`	Tabla de errores coloreada por tipo; doble clic lleva al error.
`PanelTokens` / `PanelSimbolos`	Tablas de tokens y de la tabla de símbolos.
`BarraEstado`	Estado (azul/rojo), línea y columna del cursor, tiempo de compilación.

Resaltado de sintaxis

El editor se vuelve a colorear 150 ms después de que el usuario deja de escribir. RichTextFX solo aplica estilos: la decisión de qué color lleva cada fragmento la toma el `Lexer` generado por JFlex, por lo que los colores corresponden exactamente a los tokens del lenguaje. Los colores están en `estilos.css` (clases `tk-directiva`, `tk-reservada`, `tk-comando`, `tk-conector`, `tk-cadena`, `tk-numero`, `tk-comentario`, `tk-error`).

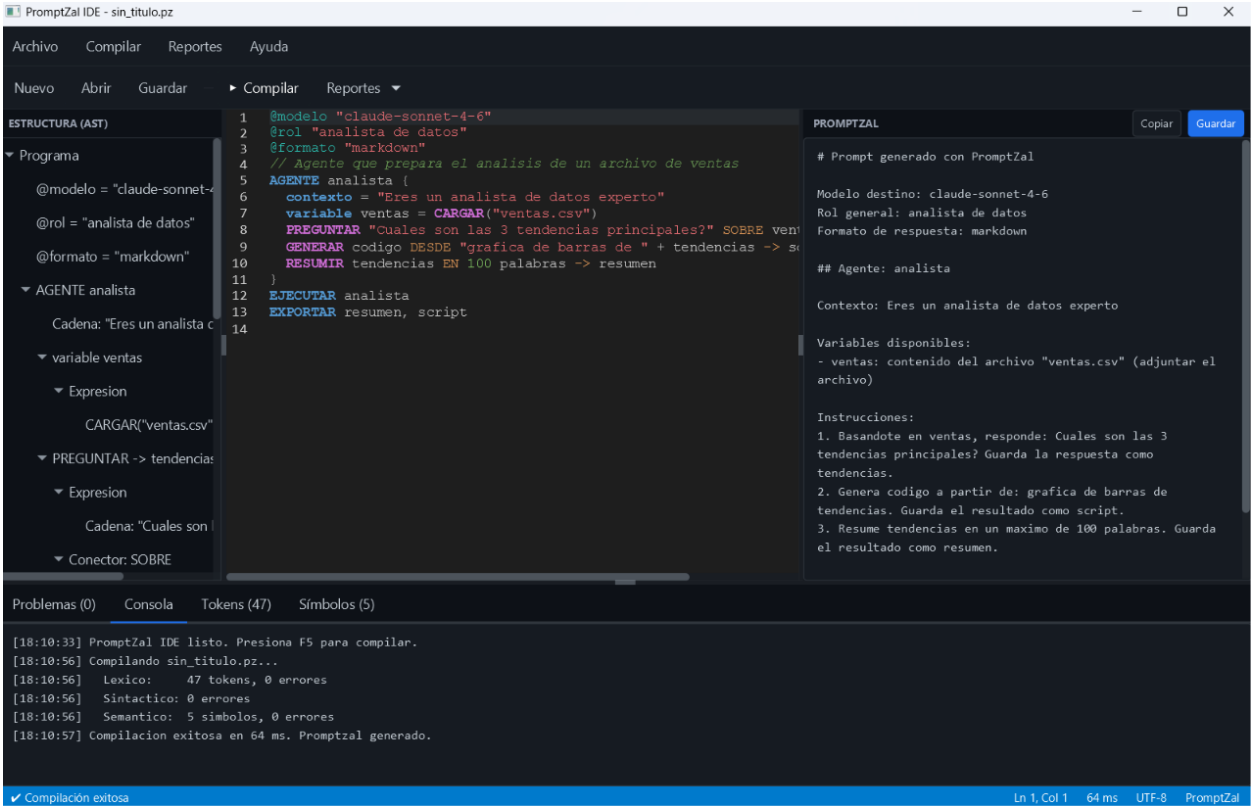


Figura 6. Interfaz principal del IDE.

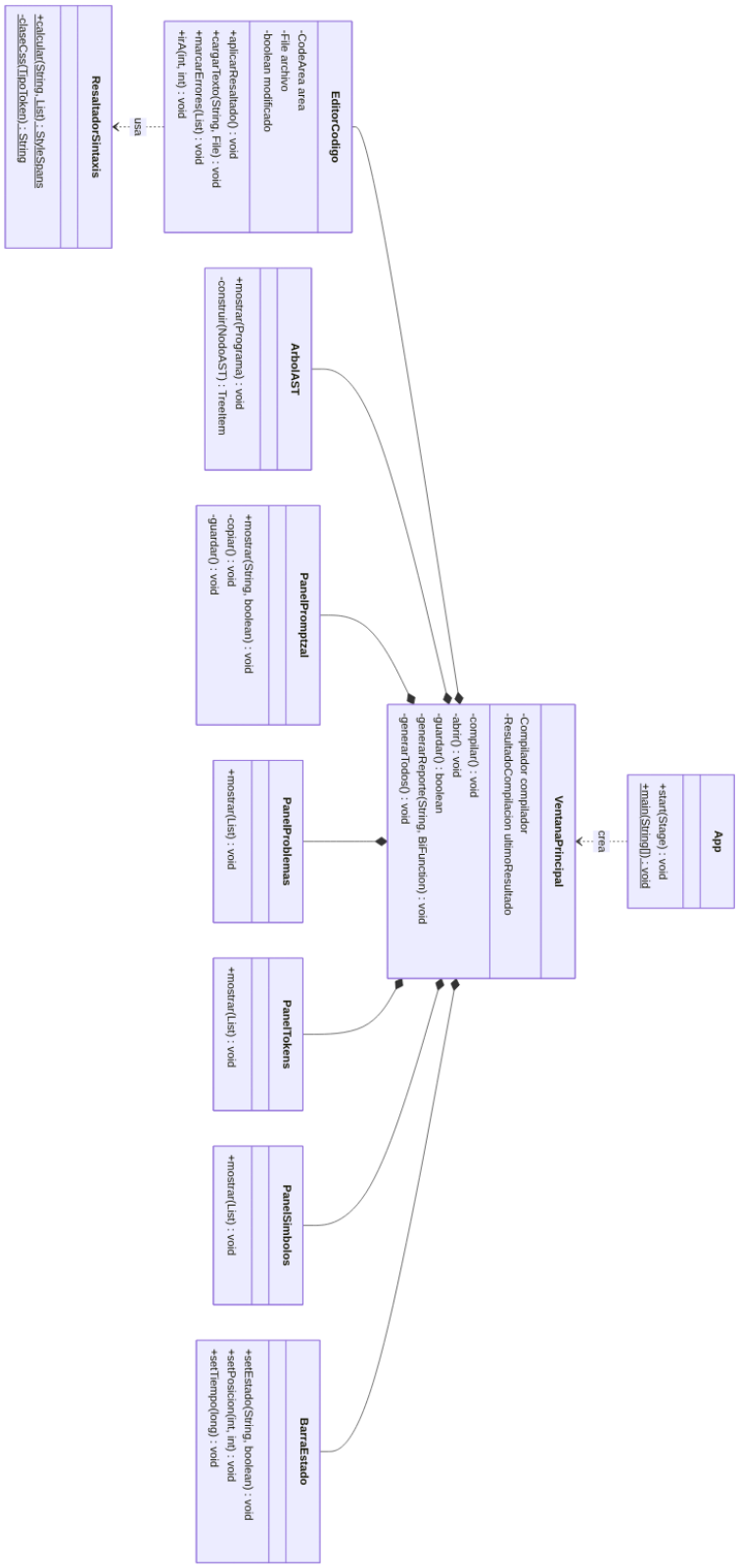


Figura 7. Diagrama de clases: interfaz gráfica.

11. Reportes

Reporte	Archivo	Contenido
Errores	`<archivo>_errores.html`	Totales por fase y tabla: tipo, lexema, descripción, fila, columna. Indica explícitamente si no hay errores.
Tabla de símbolos	`<archivo>_simbolos.html`	Totales por tipo y tabla: nombre, tipo, ámbito, fila, columna.
AST	`<archivo>_ast.html`	Árbol con líneas de conexión; cada nodo coloreado según su clase, con leyenda.
Tokens	`<archivo>_tokens.html`	Totales por categoría y tabla coloreada: número, lexema, tipo, fila, columna.

`PlantillaHTML.escapar()` convierte `<`, `>`, `&` y comillas para que el texto del usuario no rompa el HTML. Los colores de los tokens se definen con variables CSS en un solo lugar y coinciden con los del editor.

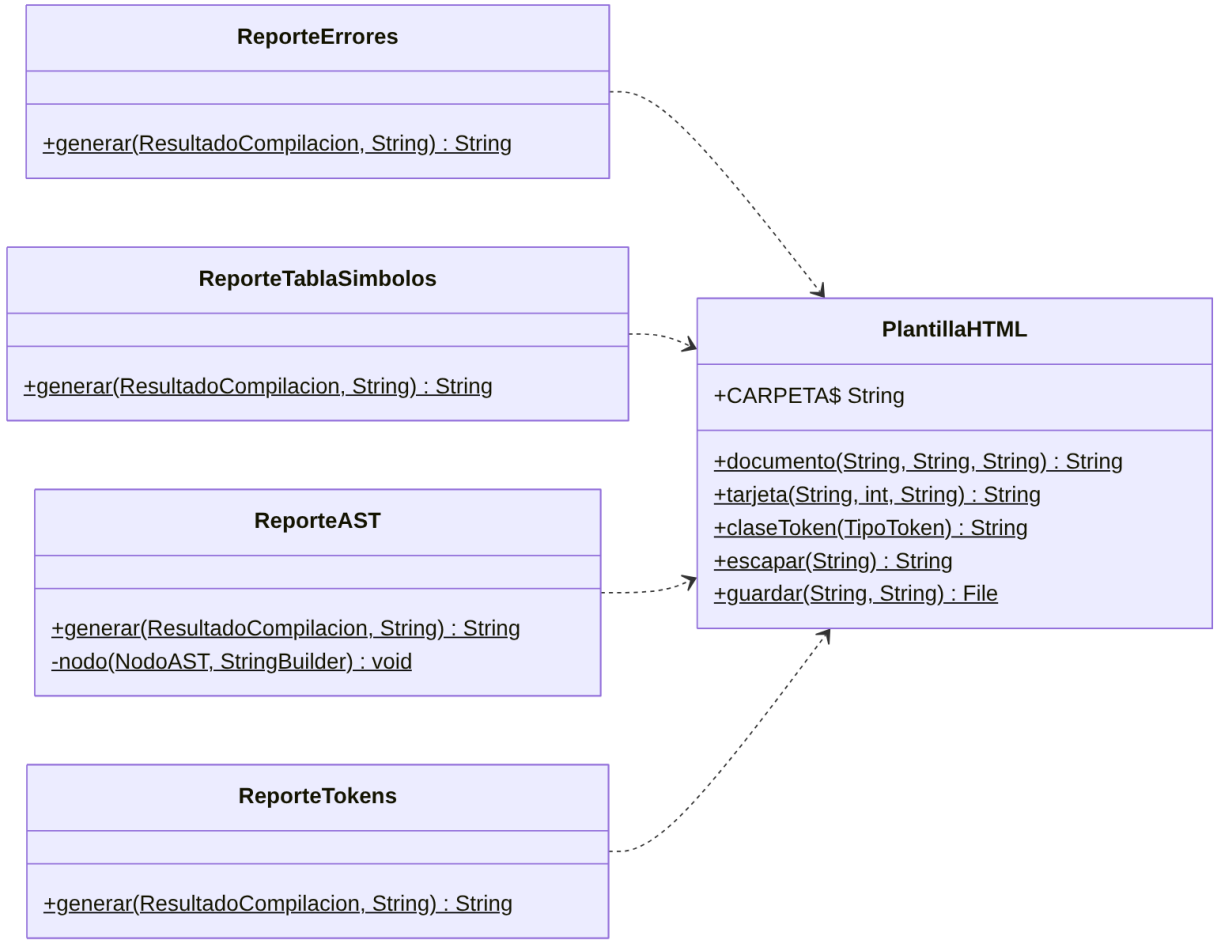


Figura 8. Diagrama de clases: reportes.

12. Casos de prueba

Archivo	Tipo	Qué verifica
`01_analista_ventas.pz`	Válido	Programa de referencia: directivas, CARGAR, concatenación, GENERAR y unidad numérica.
`02_traductor.pz`	Válido	Variable con literal y TRADUCIR con EN.
`03_moderador.pz`	Válido	CLASIFICAR con categorías y RESUMIR con límite de palabras.
`04_ingeniero.pz`	Válido	Comentario de bloque, GENERAR y conector COMO.
`05_errores_lexicos.pz`	Errores léxicos	Directiva desconocida, caracteres inválidos, cadena y comentario sin cerrar.
`06_errores_sintacticos.pz`	Errores sintácticos	Directiva sin cadena, `= =`, falta `->`, falta `}`, EJECUTAR sin nombre (5 errores, recuperación en modo pánico).
`07_errores_semanticos.pz`	Errores semánticos	Variable no declarada, ámbito de otro agente, agente duplicado, EJECUTAR inexistente, EXPORTAR de agente no ejecutado (5 errores).

13. Guía de modificaciones

El diseño concentra cada tipo de cambio en pocos lugares:

Cambio	Archivos a modificar
Nuevo comando de IA (ej. CODIFICAR)	Agregar `| "CODIFICAR"` a la regla de comandos en `Lexer.flex`. Opcional: un `case` en `GeneradorPromptzal.instruccion()`. La gramática y el color no cambian.
Nuevo conector	Agregarlo a la regla de conectores en `Lexer.flex`; si debe tener frase propia, usarlo en `instruccion()`.
Nueva directiva	Agregarla a la regla de directivas en `Lexer.flex` y una línea en `GeneradorPromptzal.encabezado()`.
Nuevo símbolo (ej. `;`)	Valor en `TipoToken`, regla en `Lexer.flex`, uso en el `Parser` si entra en la gramática y caso en `ResaltadorSintaxis.claseCss()`.
Letras con tilde o ñ en identificadores	Macro `LETRA` en `Lexer.flex`.
Nueva validación semántica	Método nuevo en `AnalizadorSemantico`, llamado desde `analizar()`.
Cambiar un color	Editor: `estilos.css`. Reportes: variables `--c` en `PlantillaHTML`.
Cambiar la redacción del prompt	`GeneradorPromptzal`: cada sección es un método.

> **Recordatorio:** después de modificar `Lexer.flex` siempre se debe hacer **Clean and Build** para regenerar `Lexer.java`.

Anexo: código Mermaid de los diagramas

Los diagramas de este manual se generaron con Mermaid. El código fuente se entrega en el archivo `diagramas_mermaid.md` y puede editarse y volver a exportarse en mermaid.live (Actions → PNG / SVG).

Figura	Diagrama
Paquetes	Dependencias entre los paquetes del proyecto
Lexer y errores	Token, TipoToken, Lexer, ManejadorErrores, ErrorCompilacion, TipoError
Parser	Métodos del parser y sus dependencias
AST	Jerarquía de NodoAST y composición del árbol
Compilador, semántico y generador	Compilador, ResultadoCompilacion, tabla de símbolos y generador
Interfaz gráfica	Clases del paquete gui
Reportes	PlantillaHTML y reportes